LIFE OS

Information & Knowledge Model

Core conceptual architecture - Checkpoint v0.1

Purpose Descrivere il modello informativo emergente: come informazione libera, proprieta, entita, relazioni, Concept e Knowledge possono crescere senza imporre struttura prematura.

Status Working model

Reading mode Core conceptual architecture

Canonical sources Master Extraction v0.1 §§17-30, 35, 40, 58-59, 100, 109-110, 124-128

Checkpoint boundary Pre-Ontology Minimum v0.1

This document records the current model at checkpoint v0.1. Working and Open material must not be read as finalized ontology.

0. Model status and scope

WORKING	La direzione compositiva e fortemente supportata dai case study, ma Entity, Property/Literal, Relation e Capability devono ancora essere formalizzati nell Ontology Minimum v0.1.

Obiettivo: ottenere poche primitive capaci di sostenere crescita progressiva, riuso, contestualita e derivazione, evitando una tassonomia con decine di tipi rigidi.

0.1 Responsibilities supported

- SR01 Whole-life map
- SR04 Idea preservation and evaluation
- SR06 Low-friction resume
- SR09 Debt and imbalance detection
- SR10 Synergy surfacing
- SR11 Emerging importance
- SR15 Knowledge reuse
- SR18 System maintenance awareness

0.2 Principles governing this model

- DP04 Progressive Structuring
- DP05 Complexity proportional to maturity
- DP06 Derive, don't duplicate
- DP07 One reality, many views
- DP08 Formalization must earn its cost
- DP13 User-controlled promotion
- DP14 Avoid premature complexity

1. The information problem

La stessa cosa puo nascere come una frase, diventare una proprieta strutturata, acquisire relazioni proprie e infine meritare identita autonoma. Un modello che obbliga a scegliere il tipo definitivo al momento della cattura crea attrito e predice male il futuro.

Design objective Permettere alla struttura di comparire quando la complessita vissuta la rende utile, senza perdere la continuita con il contenuto originario.

2. Progressive Structuring

Growth path (not a mandatory workflow)

```
Loose text
   -> Property / simple value
      -> Structured value
         -> Entity
            -> Network of entities and relations
```

- Un appunto puo restare testo per sempre se non necessita altre operazioni.
- Una proprieta puo acquisire sotto-struttura senza diventare immediatamente Entity.
- La promozione a Entity e utile quando servono identita, riuso, link indipendenti, query o lifecycle.
- La promozione deve essere controllabile dall utente; il sistema puo suggerirla come specchio.

ESTABLISHED	La progressive structuring e un principio del sistema. La forma tecnica della promotion resta Open.

3. Raw / loose information

Il primo livello deve tollerare informazione non strutturata: testo, note, immagini, screenshot, frammenti, link, commenti. L assenza di schema non e un errore: e una modalita legittima di rappresentare cose immature, emotive o non ancora classificabili.

OPEN	Il termine tecnico e la rappresentazione di Literal/raw content non sono ancora definiti. Questo e uno dei primi problemi dell Ontology v0.1.

4. Property / simple value

Una Property associa a una Entity un valore utile senza richiedere identita autonoma. Esempio: Solducci -> goal = "creare un app utile per gestire risorse condivise".

Item	Meaning
Simple value	Testo, numero, data, flag o altro valore che non richiede navigazione autonoma.
Structured value	Valore con parti interne sufficientemente utili da meritare struttura, ma non necessariamente identita globale.
Promotion candidate	Valore che inizia a essere referenziato, approfondito o riusato indipendentemente.

OPEN	Non e ancora deciso se Property sia una primitiva distinta o syntactic sugar per Relation verso un Literal/value node.

5. Entity

Working definition Qualcosa che ha sufficiente identita da poter essere riferito, ritrovato, collegato e potenzialmente approfondito autonomamente.

5.1 Promotion heuristics

- Independent reuse
- Multiple incoming links
- Multiple outgoing relations
- Substantial nested structure
- Independent lifecycle or status
- Independent operations
- Need for cross-context references
- Independent query/resume value

Nessuna soglia quantitativa universale. La domanda e se l identita autonoma riduce costo cognitivo o abilita comportamenti utili.

6. Relation

Le relazioni sono il layer connettivo che permette di ottenere molte strutture con poche primitive. Possono esprimere composizione, tipo, dipendenza, applicazione, influenza, uso, resonance e altri significati.

```
[Solducci] -> is -> [App Mobile]
[App Mobile] -> requires -> [Regulations]
[Regulations] -> composed-of -> [GDPR]
[Friedrich] -> contains -> [F2L]
```

```
[Idea] -> influences -> [Feature]
```

6.1 Relation metadata and contextual state

Una relazione puo richiedere stato contestuale proprio. GDPR non e globalmente "blocking"; puo esserlo il rapporto GDPR-per-Solducci in una certa fase.

Item	Meaning
relevance	Quanto la relazione e importante nel contesto.
status	Addressed, partial, blocker, unresolved, etc.
decision/context	Decisioni o note valide solo per quell applicazione.
time	Eventuale validita o evoluzione temporale.

WORKING Relation metadata e fortemente motivata. Quanto una Relation debba essere first-class resta da formalizzare.

7. Concept and Knowledge

Concept Entity che rappresenta qualcosa che conosco, sto comprendendo o posso approfondire.

Knowledge Famiglia/rete organizzata di Concept, contenuti e relazioni che costituisce la comprensione personale di un dominio.

Knowledge non deve necessariamente essere un contenitore che duplica i suoi componenti. Il grafo stesso puo costituire la struttura di conoscenza.

```
[Rubik's Cube]
   -> has-method -> [Friedrich]
        -> step -> [F2L]
        -> step -> [OLL]
        -> step -> [PLL]
```

WORKING Concept e un candidate type forte. Knowledge come rete/famiglia deve essere stress-testata nell ontologia.

8. Generic knowledge vs contextual application

Una conoscenza generica deve poter vivere indipendentemente dai progetti che la usano; il contesto specifico deve essere espresso nell applicazione o relazione.

```
[Solducci] -> is -> [App Mobile]
[App Mobile] -> requires -> [Regulations]
[Regulations] -> composed-of -> [GDPR]

Contextual layer:
[Solducci] <-> [GDPR]   status: blocker / partial / decided ...
```

- Evita duplicazione di GDPR dentro ogni progetto.
- Permette di migliorare la conoscenza generica una volta sola.

- Permette stati differenti per progetti differenti.
- Rende possibile derivare conoscenza rilevante da classificazioni generiche.

9. Composition and multiple lenses

La composizione deve consentire allo stesso Concept di apparire in strutture differenti. F2L puo essere parte di piu metodi e ricevere interpretazioni diverse senza essere duplicato.

Lens	Projection
CFOP/Friedrich lens	Evidenzia algoritmi rilevanti per speed, recognition e quality.
Layer-method lens	Evidenzia il subset necessario per raggiungere un certo stato.
Personal learning lens	Evidenzia cio che conosco, cio che sto studiando e i gap correnti.

10. Resources and knowledge inputs

Resource e un oggetto informativo esterno che puo alimentare Knowledge: libro, articolo, video, paper, sito, corso, immagine, screenshot, documento.

Il sistema deve poter rilevare debt tra consumo e distillazione, ma non imporre che ogni Resource generi una Knowledge formalizzata.

11. Derived information

Rule Non salvare cio che puo essere derivato in modo affidabile dal modello corrente.

- F2L progress derivato dagli algoritmi conosciuti.
- Project completion derivata dalle parti della current vision.
- Session -> uses F2L derivabile da Session -> uses Friedrich e Friedrich -> contains F2L, quando semanticamente sicuro.
- Balance derivato da Sessions/activities e classificazioni contestuali.

Attenzione: la derivazione deve essere affidabile e spiegabile. Se il significato contestuale non e deducibile senza ambiguita, una relazione esplicita puo restare necessaria.

12. Emergent schema

Lo schema puo essere scoperto dall uso. Se molte Session di uno stesso tipo ricevono spontaneamente una proprieta "Observation", il sistema puo iniziare a suggerirla in quel contesto.

```
Repeated user behavior
   -> recurring property / relation pattern
      -> system notices pattern
         -> suggests schema/capability
            -> user accepts, ignores or revises
```

ESTABLISHED Il sistema puo proporre struttura emergente; non deve applicarla automaticamente come verita.

13. Schema -> Capability -> Operation

Operational non e una classe con un pacchetto fisso di campi. Le proprieta presenti abilitano capacita e quindi operazioni.

Property/schema	Capability	Operations
status	State transition capability	Start, complete, reopen, inspect state history
scheduled_at	Scheduling capability	Place, move or compare against calendar
duration	Slot-matching capability	Filter against available time windows
temporal placement	Temporal analysis	Aggregate, trend, compare periods
metric	Quantitative analysis	Summaries, distributions, trends
requirements/blockers	Readiness capability	Detect blocked or actionable work

WORKING La correlazione schema -> capability -> operation e una delle ipotesi piu promettenti; la sua formalizzazione e un obiettivo prioritario dell Ontology v0.1.

14. Worked examples

14.1 App Mobile / GDPR

- Solducci nasce collegato al Concept App Mobile.
- App Mobile accumula conoscenza generica su pubblicazione, normative e data handling.
- GDPR viene approfondito una volta come Concept/Knowledge.
- Il progetto conserva solo lo stato contestuale dell applicazione.

14.2 Crochet structures

- Una tecnica puo iniziare come testo dentro un idea.
- Quando diventa riutilizzabile, viene promossa a Concept/Pattern.
- Nuovi progetti possono referenziarla inline.
- Il progetto finito puo produrre nuova Knowledge generalizzata.

14.3 F2L

- Concept riusabile in piu metodi.
- Composto da algoritmi.
- Mastery/progress puo essere derivato dalle parti.
- La lens corrente determina quali aspetti mostrare.

15. Design heuristics

- Prima di creare un tipo, chiedere quale operazione distinta abilita.
- Prima di creare una Entity, chiedere cosa si perde lasciandola Property/testo.
- Prima di duplicare un dato, verificare se e derivabile.
- Prima di mettere stato su Knowledge generica, verificare se appartiene invece alla relazione contestuale.
- Prima di definire schema obbligatorio, osservare quali proprieta emergono davvero.

Open questions

- Property e una primitiva o syntactic sugar per Relation + Literal?
- Come rappresentare Literal/raw content senza introdurre nodi inutili?
- Quanto deve essere first-class una Relation e quando puo acquisire identita propria?
- Come formalizzare promotion preservando riferimenti e storia?
- Come rappresentare schema/capability/operation e dove vive la regola che li collega?
- Knowledge e un profilo di Entity, una rete derivata o entrambe le cose?
- Come definire inference e derivation senza introdurre risultati opachi o ridondanti?